

// Jivops

Python para IA

Guía completa — 30 lecciones

Fundamentos · NumPy · Pandas · PyTorch · APIs de IA · Buenas prácticas y
producción

Contenido

Módulo 1: Fundamentos para ciencia de datos

- Lección 01 — Por qué Python domina el ecosistema de IA y ML
- Lección 02 — Entornos virtuales y gestión de dependencias para proyectos de IA
- Lección 03 — Jupyter Notebooks — desarrollo interactivo para IA
- Lección 04 — Tipado con type hints en código de IA
- Lección 05 — Gestores de paquetes modernos (uv, poetry) para proyectos de ML

Módulo 2: NumPy — computación numérica

- Lección 06 — Arrays de NumPy — la base de la computación numérica en Python
- Lección 07 — Broadcasting en NumPy
- Lección 08 — Operaciones vectorizadas vs bucles — por qué importa el rendimiento
- Lección 09 — Álgebra lineal con NumPy — vectores y matrices
- Lección 10 — Tipos de datos y precisión numérica en NumPy

Módulo 3: Pandas — manipulación de datos

- Lección 11 — DataFrames de Pandas — estructura central para datos tabulares
- Lección 12 — Limpieza y preprocesamiento de datos con Pandas
- Lección 13 — Agrupación y agregación de datos (groupby)
- Lección 14 — Combinar datasets — merge, join y concat
- Lección 15 — Series temporales con Pandas

Módulo 4: PyTorch — fundamentos para deep learning

- Lección 16 — Tensores de PyTorch — la estructura central de deep learning
- Lección 17 — Autograd — diferenciación automática en PyTorch
- Lección 18 — Construir una red neuronal simple con PyTorch
- Lección 19 — DataLoaders — alimentar datos al proceso de entrenamiento
- Lección 20 — Mover cómputo entre CPU y GPU con PyTorch

Módulo 5: Python para trabajar con APIs de IA

- Lección 21 — Consumir APIs de LLM desde Python — el patrón básico
- Lección 22 — Programación asíncrona en Python para llamadas a APIs de IA
- Lección 23 — Manejo de reintentos y rate limits en clientes de API
- Lección 24 — Streaming de respuestas con Python
- Lección 25 — Validación de datos con Pydantic para aplicaciones de IA

Módulo 6: Buenas prácticas y producción

Lección 26 — Testing de código que involucra modelos de IA

Lección 27 — Logging y debugging de pipelines de IA en Python

Lección 28 — Empaquetar un proyecto de IA para distribución

Lección 29 — Reproducibilidad — semillas aleatorias y determinismo

Lección 30 — Proyecto final: pipeline de IA en Python completo listo para producción

Módulo 1: Fundamentos para ciencia de datos

Lección 01 — Por qué Python domina el ecosistema de IA y ML

Un ecosistema maduro de librerías especializadas (NumPy, Pandas, PyTorch, entre otras) construidas sobre código optimizado en C/C++ por debajo, que combina la facilidad de uso de Python con un rendimiento de cómputo numérico cercano al de lenguajes de más bajo nivel.

El código Python que escribe el desarrollador actúa principalmente como una interfaz sencilla de usar, mientras el cómputo numérico realmente intensivo se delega internamente hacia la implementación optimizada en C que corre por debajo; esa separación permite combinar la facilidad de escritura de Python con un rendimiento de ejecución mucho más cercano al de un lenguaje de bajo nivel.

Lección 02 — Entornos virtuales y gestión de dependencias para proyectos de IA

Distintos proyectos de IA pueden requerir versiones específicas y a veces incompatibles entre sí de la misma librería (como una versión distinta de PyTorch), y un entorno virtual aislado evita que esas dependencias entren en conflicto entre proyectos distintos en la misma máquina.

Sin entornos virtuales separados, ambos proyectos compartirían la misma instalación global de Python y sus librerías; si un proyecto necesita la versión 1.x de PyTorch y el otro necesita la versión 2.x, instalar una sobrescribiría o entraría en conflicto directo con la otra, haciendo imposible tener ambos proyectos funcionando correctamente al mismo tiempo en esa misma máquina sin ese aislamiento.

Lección 03 — Jupyter Notebooks — desarrollo interactivo para IA

Permite ejecutar el código en celdas individuales de forma incremental, inspeccionando resultados intermedios (como una tabla de datos o un gráfico) en cada paso, sin tener que volver a ejecutar todo el script completo desde el inicio cada vez que se hace un cambio pequeño.

Cuando se explora un dataset desconocido, cada resultado revela información que influye en qué investigar a continuación (por ejemplo, ver valores faltantes podría llevar a decidir cómo limpiarlos antes de seguir); ejecutar celda por celda permite ajustar el siguiente paso basándose en lo que se acaba de descubrir, algo que un script completo ejecutado de una sola vez sin pausas no permitiría de la misma forma interactiva.

Lección 04 — Tipado con type hints en código de IA

Documenta explícitamente qué tipo de datos espera y devuelve la función, ayudando a otros desarrolladores (o al mismo autor después) a entender la interfaz sin tener que leer toda la implementación, además de permitir que herramientas de análisis estático detecten errores de tipo antes de ejecutar el código.

Al ver la firma de la función con sus type hints, un desarrollador nuevo puede entender inmediatamente qué tipo de dato debe pasar y qué recibirá de vuelta, sin tener que leer y entender toda la lógica interna de esa función para inferir esa misma información; esa documentación explícita en la propia firma ahorra tiempo real de comprensión al integrar esa función desde otra parte del proyecto.

Lección 05 — Gestores de paquetes modernos (uv, poetry) para proyectos de ML

Resuelve de forma más robusta y determinista el árbol completo de dependencias (incluidas las dependencias de las dependencias), evitando conflictos de versiones incompatibles y generando un archivo de bloqueo (lock file) que garantiza instalaciones reproducibles exactamente iguales entre distintas máquinas.

Un requirements.txt simple normalmente especifica solo las dependencias directas del proyecto, sin fijar explícitamente las versiones de las dependencias secundarias (las que esas librerías directas necesitan internamente); sin un lock file robusto que congele todo el árbol completo de dependencias, cada desarrollador podría terminar con una versión distinta de alguna dependencia secundaria simplemente según el momento en que instaló, potencialmente causando comportamiento inconsistente entre ambas máquinas.

Módulo 2: NumPy — computación numérica

Lección 06 — Arrays de NumPy — la base de la computación numérica en Python

Un array de NumPy almacena los elementos en un bloque contiguo de memoria con un tipo de dato fijo y uniforme, permitiendo operaciones numéricas mucho más rápidas y eficientes que una lista de Python, que almacena referencias a objetos dispersos y puede mezclar tipos distintos.

Cuando los datos están almacenados en un bloque contiguo de memoria, el procesador puede acceder a ellos de forma secuencial y eficiente, aprovechando optimizaciones de hardware; una lista de Python, en cambio, almacena referencias que apuntan a objetos individuales dispersos en distintas ubicaciones de memoria, requiriendo seguir cada puntero por separado, lo que introduce overhead adicional que ralentiza el procesamiento numérico comparado con el array contiguo de NumPy.

Lección 07 — Broadcasting en NumPy

Permite realizar la operación elemento por elemento entre arrays de formas distintas sin tener que escribir manualmente un bucle explícito, expandiendo automáticamente y de forma implícita el array más pequeño para que coincida con la forma del array más grande durante la operación.

Escribir `array + 5` con broadcasting expresa la intención de forma directa y concisa, delegando internamente esa operación repetitiva hacia el código optimizado de NumPy escrito en C, mientras un bucle explícito en Python puro tendría que iterar elemento por elemento dentro del intérprete de Python (más lento), además de requerir escribir más líneas de código para lograr exactamente el mismo resultado final.

Lección 08 — Operaciones vectorizadas vs bucles — por qué importa el rendimiento

Las operaciones vectorizadas delegan el procesamiento hacia código optimizado de bajo nivel que opera sobre bloques de datos de una vez, evitando el overhead que tiene el intérprete de Python al ejecutar un bucle elemento por elemento, resultando en una diferencia de rendimiento significativa especialmente con datasets grandes.

Cada iteración de un bucle 'for' en Python tiene un overhead propio del intérprete al procesar esa iteración individual; con un millón de elementos, ese overhead se repite un millón de veces, acumulando un tiempo total significativo, mientras la operación vectorizada de NumPy delega todo el procesamiento hacia código compilado que maneja el bloque completo de datos de una sola vez, sin repetir ese overhead por cada elemento individual.

Lección 09 — Álgebra lineal con NumPy — vectores y matrices

Muchas operaciones centrales de los modelos de ML (como las transformaciones dentro de una red neuronal) se expresan matemáticamente como multiplicaciones de matrices, así que dominar estas operaciones con NumPy es una base práctica directamente aplicable a entender e implementar cómo funcionan esos modelos por dentro.

Una capa de una red neuronal internamente realiza, en su núcleo, una operación de multiplicación de matrices entre los datos de entrada y los pesos aprendidos de esa capa; entender cómo funciona esa operación matemática básica con NumPy conecta directamente ese conocimiento con lo que realmente está ocurriendo dentro del modelo, en vez de tratarlo como un proceso completamente opaco sin ninguna

comprensión de su mecanismo interno real.

Lección 10 — Tipos de datos y precisión numérica en NumPy

Un tipo de menor precisión (float32) ocupa menos memoria y puede procesarse más rápido, a cambio de representar los números con menos decimales de exactitud, mientras uno de mayor precisión (float64) es más exacto pero consume más memoria y puede ser más lento de procesar.

Con un modelo que tiene millones o miles de millones de parámetros, la diferencia de memoria entre float32 y float64 se multiplica enormemente; aceptar una pequeña pérdida de exactitud numérica con float32 a cambio de usar menos memoria y procesar más rápido es un tradeoff deliberado que, para el objetivo práctico de entrenar modelos a gran escala, suele valer la pena frente a la exactitud adicional pero más costosa de float64.

Módulo 3: Pandas — manipulación de datos

Lección 11 — DataFrames de Pandas — estructura central para datos tabulares

Representa una tabla de datos bidimensional con filas y columnas etiquetadas (similar a una hoja de cálculo o una tabla de base de datos), ofreciendo operaciones integradas para filtrar, transformar y analizar esos datos de forma mucho más expresiva que manipular listas o arrays sin esa estructura.

Referirse a una columna por su nombre descriptivo como 'precio' es mucho más claro e intuitivo que tener que recordar que esa información está en la posición numérica 3 de un array sin etiquetas; ese etiquetado descriptivo reduce errores al seleccionar la columna correcta y hace que el código refleje directamente el significado real de los datos, en vez de depender de índices numéricos abstractos sin contexto.

Lección 12 — Limpieza y preprocesamiento de datos con Pandas

Un modelo entrenado con datos sucios o inconsistentes (valores faltantes sin tratar, duplicados, tipos de datos incorrectos) puede aprender patrones erróneos o fallar directamente durante el entrenamiento, así que la calidad de los datos de entrada tiene un impacto directo en la calidad del modelo resultante.

Muchos algoritmos de entrenamiento simplemente no saben cómo manejar técnicamente un valor faltante y fallarían directamente con un error; si de alguna forma el entrenamiento continuara sin fallar, el modelo podría interpretar esos valores faltantes de una forma no intencionada (como si fueran ceros u otro valor arbitrario), aprendiendo un patrón distorsionado silenciosamente que no se notaría hasta que el modelo ya está en uso y produce predicciones incorrectas sin una causa obvia.

Lección 13 — Agrupación y agregación de datos (groupby)

Divide el DataFrame en subgrupos según los valores únicos de una o más columnas especificadas, y permite aplicar una función de agregación (como suma, promedio, conteo) a cada subgrupo por separado, consolidando el resultado en una tabla resumida.

Con groupby, una sola línea de código expresa la intención completa de 'agrupar por categoría y sumar', mientras un bucle manual requeriría identificar cada categoría distinta presente en los datos y escribir lógica repetitiva de filtrado y suma para cada una por separado; groupby escala automáticamente sin importar cuántas categorías distintas existan, sin necesitar ese código repetitivo adicional.

Lección 14 — Combinar datasets — merge, join y concat

Concat apila los DataFrames uno encima del otro (o uno al lado del otro) sin necesitar una relación lógica entre sus filas, mientras merge combina las filas de ambos DataFrames basándose en una columna clave compartida, similar a un JOIN de una base de datos relacional.

Cientes y pedidos tienen una relación lógica real (cada pedido pertenece a un cliente específico identificado por un ID compartido), así que merge aprovecha esa relación para combinar correctamente la información relacionada de ambas tablas; en cambio, las ventas de enero y febrero simplemente comparten la misma estructura de columnas sin necesitar ninguna relación lógica entre filas específicas, por lo que concat (apilar uno tras otro) es la operación apropiada para ese caso distinto.

Lección 15 — Series temporales con Pandas

Permite realizar operaciones específicas de series temporales como remuestreo (agregar datos por día, semana o mes), cálculo de ventanas móviles, y selección de rangos de fechas de forma intuitiva,

aprovechando un índice de tipo fecha con comportamiento especializado.

Remuestrear con Pandas maneja automáticamente detalles complejos del calendario (meses de distinta duración, años bisiestos, zonas horarias) que serían tediosos y propensos a error de manejar manualmente escribiendo lógica de agrupación por fecha desde cero; la funcionalidad especializada ya resuelve esos casos de forma confiable, ahorrando tiempo de desarrollo y reduciendo el riesgo de errores en esa lógica de fechas escrita a mano.

Módulo 4: PyTorch — fundamentos para deep learning

Lección 16 — Tensores de PyTorch — la estructura central de deep learning

Un tensor de PyTorch es conceptualmente similar a un array de NumPy (una estructura de datos multidimensional), pero añade capacidades adicionales específicas para deep learning, como poder ejecutarse en GPU y participar en el cálculo automático de gradientes durante el entrenamiento.

Entrenar una red neuronal requiere calcular repetidamente los gradientes (derivadas) para ajustar los pesos del modelo en cada paso de aprendizaje, algo que NumPy no tiene integrado de forma automática; los tensores de PyTorch sí tienen esa capacidad incorporada (autograd), además de poder ejecutar ese cómputo intensivo en GPU para acelerarlo significativamente, dos capacidades que juntas hacen posible entrenar modelos de deep learning de forma práctica.

Lección 17 — Autograd — diferenciación automática en PyTorch

Calcula automáticamente las derivadas (gradientes) necesarias para ajustar los pesos del modelo durante el entrenamiento, rastreando las operaciones realizadas sobre los tensores para poder aplicar la regla de la cadena y propagar esos gradientes hacia atrás sin que el desarrollador tenga que calcularlos manualmente.

En una red con muchas capas, calcular manualmente cómo cada peso individual de cada capa afecta el resultado final requeriría aplicar la regla de la cadena repetidamente a través de toda esa cadena de operaciones, un trabajo matemático extenso, tedioso y muy propenso a errores humanos; autograd automatiza ese proceso completo de forma confiable, haciendo práctico entrenar arquitecturas profundas que serían virtualmente imposibles de derivar manualmente a mano.

Lección 18 — Construir una red neuronal simple con PyTorch

Se define una clase que hereda de `nn.Module`, especificando las capas del modelo en el constructor y definiendo el flujo de datos hacia adelante (forward pass) en un método específico, describiendo así tanto la estructura como el comportamiento del modelo de forma organizada.

Tener las capas declaradas claramente en un lugar (el constructor) y la lógica de cómo se conectan y usan en otro lugar separado (el método forward) hace que sea más fácil entender de un vistazo qué componentes tiene el modelo y, por separado, cómo fluyen realmente los datos a través de ellos; esa organización clara facilita modificar la arquitectura después sin tener que desenredar una mezcla confusa de declaración y uso combinados en un mismo lugar.

Lección 19 — DataLoaders — alimentar datos al proceso de entrenamiento

Divide automáticamente el dataset completo en lotes (batches) de tamaño configurable, gestiona el mezclado aleatorio de los datos entre épocas de entrenamiento, y puede cargar los datos en paralelo usando múltiples procesos para no ralentizar el entrenamiento esperando a que los datos estén listos.

Sin carga paralela, la GPU tendría que quedarse ociosa esperando mientras se prepara secuencialmente el siguiente lote de datos, antes de poder continuar procesando; con carga paralela usando múltiples procesos, el siguiente lote se prepara en segundo plano mientras la GPU todavía está ocupada procesando el lote actual, evitando ese tiempo muerto y manteniendo el entrenamiento corriendo de forma mucho más eficiente y continua.

Lección 20 — Mover cómputo entre CPU y GPU con PyTorch

Todos los tensores involucrados en una misma operación (como los datos de entrada y los pesos del modelo) deben estar en el mismo dispositivo (todos en CPU o todos en GPU) simultáneamente, ya que PyTorch no puede combinar directamente tensores ubicados en dispositivos distintos dentro de la misma operación.

La GPU y la CPU son dispositivos de cómputo físicamente separados, cada uno con su propia memoria independiente; un tensor almacenado en la memoria de la GPU no puede combinarse directamente en una operación con un tensor almacenado en la memoria de la CPU sin antes moverlos explícitamente al mismo dispositivo, así que PyTorch genera un error claro que alerta al desarrollador sobre esa inconsistencia, en vez de intentar manejarla silenciosamente de alguna forma ambigua.

Módulo 5: Python para trabajar con APIs de IA

Lección 21 — Consumir APIs de LLM desde Python — el patrón básico

Instalar el SDK oficial del proveedor, autenticar con una clave de API, construir la petición con el prompt y los parámetros deseados, enviarla, y procesar la respuesta estructurada que devuelve el servicio.

Construir manualmente las peticiones HTTP hacia la API requeriría conocer y replicar exactamente el formato específico que espera ese servicio (encabezados, estructura del cuerpo de la petición, manejo de la respuesta); el SDK oficial ya abstrae toda esa complejidad técnica, ofreciendo funciones simples y directas que logran el mismo resultado con mucho menos código y menor riesgo de errores de formato.

Lección 22 — Programación asíncrona en Python para llamadas a APIs de IA

Permite que el programa inicie varias peticiones a la API y continúe haciendo otro trabajo útil mientras espera las respuestas, en vez de bloquearse esperando cada respuesta una por una de forma completamente secuencial, mejorando significativamente el throughput total cuando hay múltiples peticiones independientes.

De forma síncrona, el tiempo total sería aproximadamente la suma del tiempo de espera de las 10 peticiones individuales, una tras otra; de forma asíncrona, varias peticiones se inician a la vez y sus tiempos de espera se superponen en vez de sumarse secuencialmente, así que el tiempo total se acerca mucho más al tiempo de la petición individual más lenta, en vez de la suma acumulada de las diez esperas completas.

Lección 23 — Manejo de reintentos y rate limits en clientes de API

Un error temporal (como un rate limit) suele resolverse por sí solo después de un breve período de espera, y esperar progresivamente más tiempo entre reintentos sucesivos evita sobrecargar aún más al servicio con peticiones repetidas inmediatas, mientras sigue dando al programa la oportunidad de recuperarse automáticamente sin intervención manual.

Un error de rate limit indica que el servicio ya está recibiendo más peticiones de las que puede procesar en ese momento; reintentar inmediatamente sin ninguna espera añadiría todavía más presión sobre ese mismo servicio ya sobrecargado, probablemente generando el mismo error de nuevo de inmediato, mientras esperar progresivamente más tiempo entre reintentos le da al servicio una oportunidad real de recuperarse antes de que el programa intente otra vez.

Lección 24 — Streaming de respuestas con Python

Se itera sobre un flujo de fragmentos (chunks) que la API va enviando progresivamente a medida que el modelo genera la respuesta, procesando o mostrando cada fragmento tan pronto como llega, en vez de acumular toda la respuesta antes de hacer algo con ella.

Si el código de la aplicación itera sobre cada fragmento de texto tan pronto como llega y lo muestra de inmediato en la interfaz, el usuario ve contenido apareciendo progresivamente; si en cambio el código esperara a acumular toda la respuesta completa antes de mostrar nada, el usuario no vería ningún contenido hasta que el modelo terminara de generar la respuesta completa, perdiendo esa experiencia de aparición progresiva del texto.

Lección 25 — Validación de datos con Pydantic para aplicaciones de IA

Define un esquema explícito de los datos esperados, y valida automáticamente que los datos reales (como una respuesta JSON del modelo) cumplan con ese esquema, lanzando un error claro e informativo si algo no coincide, en vez de que un dato mal formado cause un fallo confuso más adelante en el flujo del programa.

Si Pydantic detecta inmediatamente que falta un campo esperado, el error señala exactamente dónde y por qué ocurrió el problema, justo en el punto donde entró ese dato mal formado; si en cambio ese dato incompleto se propagara silenciosamente sin ninguna validación, podría causar un error mucho más adelante en una parte completamente distinta del código, sin ninguna conexión aparente con la causa real, haciendo el diagnóstico mucho más difícil y lento de rastrear.

Módulo 6: Buenas prácticas y producción

Lección 26 — Testing de código que involucra modelos de IA

Un modelo generativo puede producir respuestas ligeramente distintas cada vez incluso con la misma entrada, así que las pruebas no pueden simplemente comparar una salida exacta esperada, requiriendo estrategias distintas como verificar propiedades generales de la respuesta o usar mocks que simulan el modelo de forma controlada.

Llamar al modelo real en cada ejecución del conjunto de pruebas generaría un costo económico real acumulado y dependería de tener conectividad externa disponible, además de que la respuesta variable del modelo dificultaría escribir pruebas predecibles; un mock controlado permite simular una respuesta específica y consistente para verificar que el resto del código maneje correctamente ese caso, aislando la lógica propia del proyecto de la variabilidad e incertidumbre del modelo real externo.

Lección 27 — Logging y debugging de pipelines de IA en Python

El estado y los datos relevantes en cada etapa intermedia del pipeline (qué se recuperó, qué prompt final se construyó, qué respuesta se recibió), permitiendo reconstruir después exactamente qué ocurrió en cada paso si algo falla o produce un resultado inesperado.

Si el pipeline transforma la pregunta original del usuario a través de varios pasos (recuperación de contexto, construcción de prompt con plantillas) antes de enviarla al modelo, el prompt final podría ser muy distinto de la pregunta original; sin registrar ese prompt final exacto, sería imposible saber qué información realmente vio el modelo al generar esa respuesta problemática, dificultando enormemente diagnosticar en qué paso específico del pipeline se originó realmente el problema.

Lección 28 — Empaquetar un proyecto de IA para distribución

Permite que otros desarrolladores instalen el proyecto y todas sus dependencias con un solo comando, en un entorno reproducible y consistente, sin tener que averiguar manualmente qué librerías específicas necesitan instalar por separado ni en qué orden o versión exacta.

Sin un archivo que declare explícitamente qué librerías específicas y en qué versiones necesita el proyecto, cada persona que intente ejecutarlo tendría que ir corrigiendo errores de importación uno por uno (ejecutar, ver qué falta, instalar, repetir), un proceso de prueba y error lento e impreciso que además podría terminar instalando versiones incorrectas o incompatibles; un empaquetado claro con dependencias ya declaradas evita por completo ese proceso tedioso.

Lección 29 — Reproducibilidad — semillas aleatorias y determinismo

Muchos procesos de ML (como la inicialización de pesos de un modelo, o la división aleatoria de datos en entrenamiento y prueba) usan generación de números aleatorios internamente, y fijar la semilla asegura que esa secuencia de números 'aleatorios' sea exactamente la misma cada vez que se ejecuta el código, permitiendo resultados reproducibles.

Sin fijar explícitamente la misma semilla aleatoria en ambas ejecuciones, cada una generaría internamente una secuencia distinta de números pseudoaleatorios (por ejemplo, para inicializar los pesos del modelo de forma distinta); esa diferencia en los valores iniciales, aunque el resto del código y los datos sean exactamente los mismos, puede llevar el proceso de entrenamiento por un camino ligeramente distinto, resultando en un modelo final con un rendimiento no exactamente idéntico entre ambas ejecuciones.

Lección 30 — Proyecto final: pipeline de IA en Python completo listo para producción

Entorno virtual y dependencias bien gestionadas + procesamiento de datos eficiente con NumPy/Pandas + integración robusta con APIs de IA (async, reintentos, validación con Pydantic) + testing apropiado para código no determinista + logging detallado + reproducibilidad garantizada.

Intenta mencionar al menos dos pilares: gestión de entorno y dependencias, integración robusta con APIs de IA (async/reintentos/Pydantic), testing para código no determinista o logging con reproducibilidad.